Inheritance in C#: The House Analogy

Quick-reference notes — interfaces, abstract classes, and concrete classes

---

The Analogy

Think of inheritance like building a house:

Interface	A checklist of things the house must have
Abstract class	A partially built house that already gives you some things
Concrete class	The finished house that can actually be used

1. Interface — “You MUST provide these”

```
interface IAnimal
{
    string Name { get; set; }
    void Eat();
}
```

A concrete class must fulfill the contract:

```
class Dog : IAnimal
{
    public string Name { get; set; }

    public void Eat()
    {
        Console.WriteLine("Dog is eating");
    }
}
```

Idea:

```
                    Interface
                        ↓
          "Here are my requirements."
                        ↓
                  Concrete class
                        ↓
        "Okay, I implemented ALL of them.
```

2. Abstract Class — “I’ll give you some things”

```
abstract class Animal
{
```

```
    public string Name;       // already provided

    public void Sleep()       // already implemented
    {
        Console.WriteLine("Sleeping");
    }

    public abstract void Eat(); // you must eventually implement
}
```

A concrete child:

```
class Dog : Animal
{
    public override void Eat()
    {
        Console.WriteLine("Dog is eating");
    }
}
```

Dog automatically gets:

```
Name
Sleep()
```

It only has to implement:

```
Eat()
```

because `Eat()` is abstract.

3. Abstract Child — “I’m not finished yet”

You can have an abstract class inherit from another abstract class:

```
abstract class Dog : Animal
{
    // Eat() is still not implemented
}
```

■ This is allowed because **Dog** is still abstract.

Later:

```
class Puppy : Dog
{
    public override void Eat()
    {
        Console.WriteLine("Puppy is eating");
    }
}
```

Now **Puppy** is concrete, so everything required is finished.

What About Attributes?

> Be careful with the word *attribute* here — in C# you usually mean **field** or **property**.

Normal field / property

```
abstract class Animal
{
    public string Name;
}
```

The child doesn't need to implement it:

```
class Dog : Animal
{
    // Name is already inherited
}
```

Because `Name` was already provided.

Abstract property

```
abstract class Animal
{
    public abstract string Name { get; set; }
}
```

A concrete child must implement it:

```
class Dog : Animal
{
    public override string Name { get; set; }
}
```

■ The Main Rule

```
Interface
    ↓
"You must fulfill my contract"
    ↓
Concrete class
MUST implement it
```

```
Abstract class
    ↓
"I'll give you some things,
but some things are unfinished"
    ↓
Abstract members → MUST
eventually be implemented
```

And: an abstract child can leave unfinished members for its own children.

Exam Memory Trick

```
Interface    = WHAT you must do
Abstract class = WHAT you already have + WHAT you still must do
Concrete class = EVERYTHING IS FINISHED
```